

Page Replacement Algorithms

1 Core Idea

Definition

Page replacement is the operating system's decision about which page to evict from physical memory when a page fault occurs and a free page frame is needed.

When a process references a page that is not currently in RAM, a **page fault** occurs. The operating system must bring the missing page into memory. If no free frame is available, some existing page must be removed.

Basic Fault Handling Idea

page fault → choose victim page → evict victim → load needed page

2 Why Replacement Is Needed

Physical memory is limited. Virtual memory lets processes have large address spaces, but only some pages can be in RAM at one time.

Situation	What the OS must do
Free frame exists	Load the missing page into the free frame.
No free frame exists	Pick an existing page as the victim and evict it.
Victim page is clean	Overwrite it; disk already has a correct copy.
Victim page is dirty	Write it back to disk before reusing the frame.

Main Goal

A good replacement algorithm tries to evict a page that will not be needed again soon.

3 Clean Pages and Dirty Pages

Clean vs Dirty

A **clean page** has not been modified since it was loaded from disk. A **dirty page** has been modified while in memory.

This distinction matters because eviction cost is different.

Page state	Eviction action
Clean	No disk write is needed. The page frame can be reused directly.
Dirty	The page must be written back to disk before the frame is reused.

Why Dirty Pages Are Expensive

Evicting a dirty page requires disk I/O. Disk I/O is far slower than normal memory access, so replacement algorithms often prefer clean victims when possible.

4 Why Random Replacement Is Weak

The OS could choose a random page to evict, but this often performs poorly.

Choice	Consequence
Evict rarely used page	Good choice. The page may not be needed again for a long time.
Evict heavily used page	Bad choice. The page may fault back in almost immediately.

Performance Idea

The cost of a bad eviction is not only the current page fault. It can also create another page fault soon after.

5 What Page Replacement Algorithms Try to Predict

A replacement algorithm does not know the future exactly. It uses available information to estimate which page is least useful.

Information	Why it helps
Recent use	Recently used pages may be used again soon.
Reference bit	Shows whether a page has been accessed recently.
Modified bit	Shows whether eviction requires writing the page back to disk.
Process behavior	Some pages belong to active code, data, or stack regions.

6 Replacement in Other Systems

Page replacement is one example of a broader problem: when a storage layer is full, choose what to remove.

System	What is cached	Replacement question
Virtual memory	Disk pages in RAM	Which page frame should be reused?
CPU cache	Recent memory blocks, often 32 or 64 bytes each	Which cache block should be replaced?
Web server cache	Frequently requested web pages	Which cached web page should be removed?

Same Pattern, Different Speed

The idea is similar across systems, but the time scale differs. CPU cache replacement must happen in nanoseconds, while virtual-memory replacement may involve millisecond-scale disk I/O.

7 Clean and Dirty in Different Caches

Virtual memory has an extra issue: pages in RAM may be modified.

Cache type	Can cached item be modified?	Eviction consequence
Virtual memory	Yes	Dirty pages must be written back to disk.
Web page cache	Usually no	A fresh copy remains on disk or storage, so no write-back is needed.
CPU cache	Yes, depending on cache policy	Dirty cache blocks may need write-back to lower memory.

8 Local vs Global Replacement

When a page must be evicted, the OS has to decide the search scope.

Two Scopes

Local replacement: evict one of the faulting process's own pages.

Global replacement: evict a page from any process in memory.

Policy	Meaning	Main effect
Local replacement	The victim must belong to the process that caused the page fault.	Each process is effectively limited to its own assigned frames.
Global replacement	The victim may belong to another process.	Memory can shift toward processes that currently need more frames.

Important Design Question

Page replacement is not only about which page is old or unused. It is also about whether one process is allowed to take memory frames away from another process.

9 Page Replacement Flow

1. A process references a page.
2. The MMU discovers that the page is not present in RAM.
3. A page fault traps to the operating system.
4. The OS checks for a free frame.
5. If no free frame exists, the OS chooses a victim page.
6. If the victim is dirty, the OS writes it back to disk.
7. The OS loads the needed page into the freed frame.
8. The page table and TLB information are updated.
9. The faulting instruction is restarted.

10 What Makes a Good Victim?

Good victim property	Reason
Not recently used	Less likely to be needed again soon.
Clean	Can be evicted without disk write-back.
Belongs to inactive process	Less likely to be referenced immediately.
Not part of active code/data/stack	Lower chance of immediate refault.

Central Principle

The best victim is a page whose removal causes the least future cost.

11 Final Summary

Concept	Key point
Page fault	The needed page is not currently available in RAM.
Page replacement	The OS chooses a resident page to evict.
Clean page	Can be discarded because disk already has the correct copy.
Dirty page	Must be written back before eviction.
Good algorithm	Avoids evicting pages that will soon be used again.
Local/global choice	Determines whether the victim must come from the faulting process or from any process.

One-Box Summary

Page replacement chooses a victim page when RAM is full.
Clean victims are cheaper than dirty victims.
A good policy avoids evicting heavily used pages.
The OS must also choose local or global replacement scope.